

题目1：十进制转二进制

题目描述

将下列十进制数（含小数）转换为二进制数： - 57, 128, 12.5, 7.198, 3972, 1.35, 1000

代码实现

```
def decimal_to_binary(num):
    if num == 0:
        return "0"

    sign = ""
    if num < 0:
        sign = "-"
        num = abs(num)

    integer_part = int(num)
    fractional_part = num - integer_part

    binary_integer = []
    if integer_part > 0:
        temp = integer_part
        print(f"  整数部分转换 ({integer_part} → 二进制):")
        while temp > 0:
            remainder = temp % 2
            binary_integer.append(str(remainder))
            print(f"      {temp} ÷ 2 = {temp // 2} ... 余数 {remainder}")
            temp = temp // 2
        binary_integer.reverse()
    else:
        binary_integer = ["0"]

    binary_fractional = []
    if fractional_part > 0:
        temp = fractional_part
        print(f"  小数部分转换 ({fractional_part:.6f} → 二进制):")
        precision = 20
        count = 0
        while temp > 0 and count < precision:
            original = temp
            temp *= 2
            integer = int(temp)
            binary_fractional.append(str(integer))
            print(f"      {original:.6f} × 2 = {temp:.6f} ... 整数 {integer}")
            temp -= integer
            count += 1

    result = sign + "".join(binary_integer)
    if binary_fractional:
        result += "." + "".join(binary_fractional)

    return result
```

运行结果

【十进制】57

整数部分转换 (57 → 二进制):

57 ÷ 2 = 28 ... 余数 1
28 ÷ 2 = 14 ... 余数 0
14 ÷ 2 = 7 ... 余数 0
7 ÷ 2 = 3 ... 余数 1
3 ÷ 2 = 1 ... 余数 1
1 ÷ 2 = 0 ... 余数 1

【二进制】111001

【十进制】128

整数部分转换 (128 → 二进制):

128 ÷ 2 = 64 ... 余数 0
64 ÷ 2 = 32 ... 余数 0
32 ÷ 2 = 16 ... 余数 0
16 ÷ 2 = 8 ... 余数 0
8 ÷ 2 = 4 ... 余数 0
4 ÷ 2 = 2 ... 余数 0
2 ÷ 2 = 1 ... 余数 0
1 ÷ 2 = 0 ... 余数 1

【二进制】10000000

【十进制】12.5

整数部分转换 (12 → 二进制):

12 ÷ 2 = 6 ... 余数 0
6 ÷ 2 = 3 ... 余数 0
3 ÷ 2 = 1 ... 余数 1
1 ÷ 2 = 0 ... 余数 1

小数部分转换 (0.500000 → 二进制):

0.500000 × 2 = 1.000000 ... 整数 1

【二进制】1100.1

【十进制】7.198

整数部分转换 (7 → 二进制):

$7 \div 2 = 3 \dots \text{余数 } 1$
 $3 \div 2 = 1 \dots \text{余数 } 1$
 $1 \div 2 = 0 \dots \text{余数 } 1$

小数部分转换 (0.198000 $\rightarrow$ 二进制):

$0.198000 \times 2 = 0.396000 \dots \text{整数 } 0$
 $0.396000 \times 2 = 0.792000 \dots \text{整数 } 0$
 $0.792000 \times 2 = 1.584000 \dots \text{整数 } 1$
 $0.584000 \times 2 = 1.168000 \dots \text{整数 } 1$
 $0.168000 \times 2 = 0.336000 \dots \text{整数 } 0$
 $0.336000 \times 2 = 0.672000 \dots \text{整数 } 0$
 $0.672000 \times 2 = 1.344000 \dots \text{整数 } 1$
 $0.344000 \times 2 = 0.688000 \dots \text{整数 } 0$
 $0.688000 \times 2 = 1.376000 \dots \text{整数 } 1$
 $0.376000 \times 2 = 0.752000 \dots \text{整数 } 0$
 $0.752000 \times 2 = 1.504000 \dots \text{整数 } 1$
 $0.504000 \times 2 = 1.008000 \dots \text{整数 } 1$
 $0.008000 \times 2 = 0.016000 \dots \text{整数 } 0$
 $0.016000 \times 2 = 0.032000 \dots \text{整数 } 0$
 $0.032000 \times 2 = 0.064000 \dots \text{整数 } 0$
 $0.064000 \times 2 = 0.128000 \dots \text{整数 } 0$
 $0.128000 \times 2 = 0.256000 \dots \text{整数 } 0$
 $0.256000 \times 2 = 0.512000 \dots \text{整数 } 0$
 $0.512000 \times 2 = 1.024000 \dots \text{整数 } 1$
 $0.024000 \times 2 = 0.048000 \dots \text{整数 } 0$

【二进制】111.00110010101100000010

【十进制】3972

整数部分转换 (3972 $\rightarrow$ 二进制):

$3972 \div 2 = 1986 \dots \text{余数 } 0$
 $1986 \div 2 = 993 \dots \text{余数 } 0$
 $993 \div 2 = 496 \dots \text{余数 } 1$
 $496 \div 2 = 248 \dots \text{余数 } 0$
 $248 \div 2 = 124 \dots \text{余数 } 0$
 $124 \div 2 = 62 \dots \text{余数 } 0$
 $62 \div 2 = 31 \dots \text{余数 } 0$
 $31 \div 2 = 15 \dots \text{余数 } 1$
 $15 \div 2 = 7 \dots \text{余数 } 1$
 $7 \div 2 = 3 \dots \text{余数 } 1$
 $3 \div 2 = 1 \dots \text{余数 } 1$
 $1 \div 2 = 0 \dots \text{余数 } 1$

【二进制】111110000100

【十进制】1.35

整数部分转换 (1 $\rightarrow$ 二进制):

$1 \div 2 = 0 \dots \text{余数 } 1$

小数部分转换 (0.350000 $\rightarrow$ 二进制):

$0.350000 \times 2 = 0.700000 \dots \text{整数 } 0$
 $0.700000 \times 2 = 1.400000 \dots \text{整数 } 1$
 $0.400000 \times 2 = 0.800000 \dots \text{整数 } 0$
 $0.800000 \times 2 = 1.600000 \dots \text{整数 } 1$
 $0.600000 \times 2 = 1.200000 \dots \text{整数 } 1$
 $0.200000 \times 2 = 0.400000 \dots \text{整数 } 0$
 $0.400000 \times 2 = 0.800000 \dots \text{整数 } 0$
 $0.800000 \times 2 = 1.600000 \dots \text{整数 } 1$
 $0.600000 \times 2 = 1.200000 \dots \text{整数 } 1$
 $0.200000 \times 2 = 0.400000 \dots \text{整数 } 0$
 $0.400000 \times 2 = 0.800000 \dots \text{整数 } 0$
 $0.800000 \times 2 = 1.600000 \dots \text{整数 } 1$
 $0.600000 \times 2 = 1.200000 \dots \text{整数 } 1$
 $0.200000 \times 2 = 0.400000 \dots \text{整数 } 0$
 $0.400000 \times 2 = 0.800000 \dots \text{整数 } 0$
 $0.800000 \times 2 = 1.600000 \dots \text{整数 } 1$
 $0.600000 \times 2 = 1.200000 \dots \text{整数 } 1$
 $0.200000 \times 2 = 0.400000 \dots \text{整数 } 0$
 $0.400000 \times 2 = 0.800000 \dots \text{整数 } 0$
 $0.800000 \times 2 = 1.600000 \dots \text{整数 } 1$

【二进制】1.01011001100110011001

【十进制】1000

整数部分转换 (1000 $\rightarrow$ 二进制):

$1000 \div 2 = 500 \dots \text{余数 } 0$
 $500 \div 2 = 250 \dots \text{余数 } 0$
 $250 \div 2 = 125 \dots \text{余数 } 0$
 $125 \div 2 = 62 \dots \text{余数 } 1$
 $62 \div 2 = 31 \dots \text{余数 } 0$

31 ÷ 2 = 15 ... 余数 1
15 ÷ 2 = 7 ... 余数 1
7 ÷ 2 = 3 ... 余数 1
3 ÷ 2 = 1 ... 余数 1
1 ÷ 2 = 0 ... 余数 1

【二进制】1111101000

题目2：二进制转十进制

题目描述

将下列二进制数（含小数）转换为十进制数： - 11010, 110, 11.101, 0.1011, 111.11, 111111

代码实现

```
def binary_to_decimal(binary_str):
    if not binary_str:
        return 0

    sign = 1
    if binary_str.startswith("-"):
        sign = -1
        binary_str = binary_str[1:]

    if "." in binary_str:
        integer_part_str, fractional_part_str = binary_str.split(".", 1)
    else:
        integer_part_str = binary_str
        fractional_part_str = ""

    decimal_integer = 0
    if integer_part_str:
        print(f"  整数部分转换 ({integer_part_str} → 十进制):")
        n = len(integer_part_str)
        for i in range(n):
            digit = int(integer_part_str[i])
            exponent = n - 1 - i
            value = digit * (2 ** exponent)
            decimal_integer += value
            print(f"    {digit} × 2^{exponent} = {value}")
        print(f"  整数部分总和: {decimal_integer}")

    decimal_fractional = 0
    if fractional_part_str:
        print(f"  小数部分转换 ({fractional_part_str} → 十进制):")
        n = len(fractional_part_str)
        for i in range(n):
            digit = int(fractional_part_str[i])
            exponent = -(i + 1)
            value = digit * (2 ** exponent)
            decimal_fractional += value
            print(f"    {digit} × 2^{exponent} = {value}")
        print(f"  小数部分总和: {decimal_fractional:.6f}")

    return sign * (decimal_integer + decimal_fractional)
```

运行结果

【二进制】11010
整数部分转换 (11010 → 十进制):
1 × 2⁴ = 16
1 × 2³ = 8
0 × 2² = 0
1 × 2¹ = 2
0 × 2⁰ = 0
整数部分总和: 26

【十进制】26

【二进制】110
整数部分转换 (110 → 十进制):
1 × 2² = 4
1 × 2¹ = 2
0 × 2⁰ = 0
整数部分总和: 6

【十进制】6

【二进制】11.101
整数部分转换 (11 → 十进制):
1 × 2¹ = 2
1 × 2⁰ = 1
整数部分总和: 3
小数部分转换 (101 → 十进制):
1 × 2⁻¹ = 0.5
0 × 2⁻² = 0.0
1 × 2⁻³ = 0.125
小数部分总和: 0.625000

【十进制】3.625

【二进制】0.1011
整数部分转换 (0 → 十进制):
0 × 2⁰ = 0
整数部分总和: 0
小数部分转换 (1011 → 十进制):
1 × 2⁻¹ = 0.5
0 × 2⁻² = 0.0
1 × 2⁻³ = 0.125
1 × 2⁻⁴ = 0.0625
小数部分总和: 0.687500

【十进制】0.6875

【二进制】111.11

整数部分转换 (111 → 十进制):

$1 \times 2^2 = 4$

$1 \times 2^1 = 2$

$1 \times 2^0 = 1$

整数部分总和: 7

小数部分转换 (11 → 十进制):

$1 \times 2^{-1} = 0.5$

$1 \times 2^{-2} = 0.25$

小数部分总和: 0.750000

【十进制】7.75

【二进制】111111

整数部分转换 (111111 → 十进制):

$1 \times 2^5 = 32$

$1 \times 2^4 = 16$

$1 \times 2^3 = 8$

$1 \times 2^2 = 4$

$1 \times 2^1 = 2$

$1 \times 2^0 = 1$

整数部分总和: 63

【十进制】63

题目3: 进制转换

题目描述

将下列二进制数转换为八进制和十六进制，八或十六进制转为二进制数： - 【二进制数】101110101, 1101100.11 - 【八进制数】3756, 415.213 - 【十六进制】C6F0, 5AB.4D

代码实现

```
def binary_to_octal(binary_str):
    if "." in binary_str:
        integer_part, fractional_part = binary_str.split(".", 1)
    else:
        integer_part = binary_str
        fractional_part = ""

    print(f"  二进制整数部分分组 ({integer_part}):")
    pad_length = (3 - len(integer_part) % 3) % 3
    padded_integer = "0" * pad_length + integer_part
    print(f"    补零后: {padded_integer}")

    octal_integer = []
    for i in range(0, len(padded_integer), 3):
        group = padded_integer[i:i+3]
        octal_digit = str(int(group, 2))
        octal_integer.append(octal_digit)
        print(f"    分组 {group} → 八进制 {octal_digit}")

    octal_fractional = []
    if fractional_part:
        print(f"  二进制小数部分分组 ({fractional_part}):")
        pad_length = (3 - len(fractional_part) % 3) % 3
        padded_fractional = fractional_part + "0" * pad_length
        print(f"    补零后: {padded_fractional}")

        for i in range(0, len(padded_fractional), 3):
            group = padded_fractional[i:i+3]
            octal_digit = str(int(group, 2))
            octal_fractional.append(octal_digit)
            print(f"    分组 {group} → 八进制 {octal_digit}")

    result = "".join(octal_integer)
    if octal_fractional:
        result += "." + "".join(octal_fractional)

    return result

def binary_to_hex(binary_str):
    hex_chars = "0123456789ABCDEF"
    if "." in binary_str:
        integer_part, fractional_part = binary_str.split(".", 1)
    else:
        integer_part = binary_str
        fractional_part = ""

    print(f"  二进制整数部分分组 ({integer_part}):")
    pad_length = (4 - len(integer_part) % 4) % 4
    padded_integer = "0" * pad_length + integer_part
    print(f"    补零后: {padded_integer}")

    hex_integer = []
    for i in range(0, len(padded_integer), 4):
        group = padded_integer[i:i+4]
        hex_digit = hex_chars[int(group, 2)]
        hex_integer.append(hex_digit)
        print(f"    分组 {group} → 十六进制 {hex_digit}")

    hex_fractional = []
    if fractional_part:
        print(f"  二进制小数部分分组 ({fractional_part}):")
        pad_length = (4 - len(fractional_part) % 4) % 4
        padded_fractional = fractional_part + "0" * pad_length
        print(f"    补零后: {padded_fractional}")

        for i in range(0, len(padded_fractional), 4):
            group = padded_fractional[i:i+4]
            hex_digit = hex_chars[int(group, 2)]
            hex_fractional.append(hex_digit)
            print(f"    分组 {group} → 十六进制 {hex_digit}")

    result = "".join(hex_integer)
    if hex_fractional:
        result += "." + "".join(hex_fractional)

    return result

def octal_to_binary(octal_str):
    octal_to_bin = {
        "0": "000", "1": "001", "2": "010", "3": "011",
        "4": "100", "5": "101", "6": "110", "7": "111"
    }

    if "." in octal_str:
        integer_part, fractional_part = octal_str.split(".", 1)
    else:
        integer_part = octal_str
        fractional_part = ""

    print(f"  八进制整数部分转换 ({integer_part}):")
    binary_integer = []
    for digit in integer_part:
        binary = octal_to_bin[digit]
        binary_integer.append(binary)
        print(f"    {digit} → {binary}")

    binary_fractional = []
    if fractional_part:
        print(f"  八进制小数部分转换 ({fractional_part}):")
        for digit in fractional_part:
            binary = octal_to_bin[digit]
            binary_fractional.append(binary)
            print(f"    {digit} → {binary}")

    result = "".join(binary_integer).lstrip("0") or "0"
```

```
if binary_fractional:
    result += "." + "".join(binary_fractional).rstrip("0")

return result

def hex_to_binary(hex_str):
    hex_to_bin = {
        "0": "0000", "1": "0001", "2": "0010", "3": "0011",
        "4": "0100", "5": "0101", "6": "0110", "7": "0111",
        "8": "1000", "9": "1001", "A": "1010", "B": "1011",
        "C": "1100", "D": "1101", "E": "1110", "F": "1111",
        "a": "1010", "b": "1011", "c": "1100", "d": "1101",
        "e": "1110", "f": "1111"
    }

    if "." in hex_str:
        integer_part, fractional_part = hex_str.split(".", 1)
    else:
        integer_part = hex_str
        fractional_part = ""

    print(f" 十六进制整数部分转换 ({integer_part}):")
    binary_integer = []
    for digit in integer_part:
        binary = hex_to_bin[digit]
        binary_integer.append(binary)
    print(f"      {digit} → {binary}")

    binary_fractional = []
    if fractional_part:
        print(f" 十六进制小数部分转换 ({fractional_part}):")
        for digit in fractional_part:
            binary = hex_to_bin[digit]
            binary_fractional.append(binary)
        print(f"      {digit} → {binary}")

    result = "".join(binary_integer).lstrip("0") or "0"
    if binary_fractional:
        result += "." + "".join(binary_fractional).rstrip("0")

    return result
```

运行结果

【二进制】101110101

二进制整数部分分组 (101110101):

补零后: 101110101

分组 101 → 八进制 5

分组 110 → 八进制 6

分组 101 → 八进制 5

【八进制】565

二进制整数部分分组 (101110101):

补零后: 000101110101

分组 0001 → 十六进制 1

分组 0111 → 十六进制 7

分组 0101 → 十六进制 5

【十六进制】175

【二进制】1101100.11

二进制整数部分分组 (1101100):

补零后: 001101100

分组 001 → 八进制 1

分组 101 → 八进制 5

分组 100 → 八进制 4

二进制小数部分分组 (11):

补零后: 110

分组 110 → 八进制 6

【八进制】154.6

二进制整数部分分组 (1101100):

补零后: 01101100

分组 0110 → 十六进制 6

分组 1100 → 十六进制 C

二进制小数部分分组 (11):

补零后: 1100

分组 1100 → 十六进制 C

【十六进制】6C.C

【八进制】3756

八进制整数部分转换 (3756):

3 → 011

7 → 111

5 → 101

6 → 110

【二进制】11111101110

【八进制】415.213

八进制整数部分转换 (415):

4 → 100

1 → 001

5 → 101

八进制小数部分转换 (213):

2 → 010

1 → 001

3 → 011

【二进制】100001101.010001011

【十六进制】C6F0

十六进制整数部分转换 (C6F0):

C → 1100

6 → 0110

F → 1111

0 → 0000

【二进制】1100011011110000

【十六进制】5AB.4D

十六进制整数部分转换 (5AB):

5 → 0101

A → 1010

B → 1011

十六进制小数部分转换 (4D):

4 → 0100

D → 1101

【二进制】10110101011.01001101

题目4: Unicode编号和UTF-8编码

题目描述

查找下列字符的Unicode编号和UTF-8编码： - Python语言基础与人工智能应用

代码实现

```
def unicode_utf8_analysis(text):
    print(f"  字符 | Unicode编号 | UTF-8编码")
    print(f"{'-'*40}")
    for char in text:
        unicode_code = ord(char)
        utf8_bytes = char.encode('utf-8')
        utf8_hex = " ".join(f"{b:02X}" for b in utf8_bytes)
        print(f" {char} | U+{unicode_code:04X} | {utf8_hex}")
```

运行结果

【字符串】Python语言基础与人工智能应用

字符	Unicode编号	UTF-8编码

'P'	U+0050	50
'y'	U+0079	79
't'	U+0074	74
'h'	U+0068	68
'o'	U+006F	6F
'n'	U+006E	6E
'语'	U+8BED	E8 AF AD
'言'	U+8A00	E8 A8 80
'基'	U+57FA	E5 9F BA
'础'	U+7840	E7 A1 80
'与'	U+4E0E	E4 B8 8E
'人'	U+4EBA	E4 BA BA
'工'	U+5DE5	E5 B7 A5
'智'	U+667A	E6 99 BA
'能'	U+80FD	E8 83 BD
'应'	U+5E94	E5 BA 94
'用'	U+7528	E7 94 A8

题目5：MP3音频文件分析

题目描述

查阅Test 1 Part 1.mp3采样频率、声道、码率、时长、容量等信息，计算它相对于原始音频编码容量的压缩率。

代码实现

```
import os
from mutagen.mp3 import MP3

def analyze_mp3(file_path):
    audio = MP3(file_path)
    file_size = os.path.getsize(file_path)

    sample_rate = audio.info.sample_rate
    channels = audio.info.channels
    bitrate = audio.info.bitrate / 1000
    duration = audio.info.length

    print("文件信息：")
    print(f"  文件名: {os.path.basename(file_path)}")
    print(f"  文件格式: MP3 (MPEG-1 Audio Layer 3)")
    print(f"  采样频率: {sample_rate} Hz")
    print(f"  声道数: {channels}")
    print(f"  码率: {bitrate:.1f} kbps")
    print(f"  时长: {int(duration // 60)}分{duration % 60:.2f}秒")
    print(f"  文件容量: {file_size} 字节")

    original_pcm_size = sample_rate * channels * duration * 16 / 8
    compression_rate = original_pcm_size / file_size

    print("\n压缩率计算过程：")
    print(f"  原始PCM编码容量 = {sample_rate} × {channels} × {duration:.2f} × 16 / 8")
    print(f"                    = {original_pcm_size:.0f} 字节")
    print(f"  实际文件容量 = {file_size} 字节")
    print(f"  压缩率 = {original_pcm_size:.0f} / {file_size} = {compression_rate:.2f}")
    print(f"  压缩比 = (100 - (100 / compression_rate):.1f)%")

    return {
        'sample_rate': sample_rate,
        'channels': channels,
        'bitrate': bitrate,
        'duration': duration,
        'file_size': file_size,
        'original_pcm_size': original_pcm_size,
        'compression_rate': compression_rate
    }

result = analyze_mp3("Test 1 Part 1.mp3")
```

文件信息

属性	值
文件名	Test 1 Part 1.mp3
文件格式	MP3 (MPEG-1 Audio Layer 3)
采样频率	44100 Hz
声道数	2
码率	256.0 kbps
时长	7分02秒 (422.04秒)
文件容量	12.88 MB (13505290 字节)

压缩率计算步骤

1. 原始PCM编码容量计算公式：

原始PCM编码容量 = 采样频率 × 声道数 × 时长 × 位深 / 8

其中： - 采样频率 = 44100 Hz - 声道数 = 2 - 时长 = 422.04 秒 - 位深 = 16 bits（标准CD音质）

2. 代入数值计算：

原始PCM编码容量 = 44100 × 2 × 422.04 × 16 / 8 = 44100 × 2 × 422.04 × 2 = 44100 × 1688.16 = 74447911 字节

3. 转换为更易读的单位：

74447911 字节 ÷ 1024 ÷ 1024 ≈ 71.00 MB

4. 计算压缩率：

压缩率 = 原始PCM编码容量 / 实际文件容量 = 74447911 / 13505290 ≈ 5.51

5. 计算压缩比：

压缩比 = (1 - 实际容量 / 原始容量) × 100% = (1 - 1/5.51) × 100% ≈ 81.9%

计算结果

指标	值
原始PCM编码容量	71.00 MB
实际文件容量	12.88 MB
压缩率	5.51
压缩比	81.9%

题目6：MIDI文件分析与收听评价

题目描述

查阅圣诞节快乐劳伦斯先生.mid的时长、容量等信息，并播放收听，写出你对收听效果的评价。

代码实现

```
import os
from mido import MidiFile

def analyze_midi(file_path):
    mid = MidiFile(file_path)
    file_size = os.path.getsize(file_path)

    duration = mid.length
    ticks_per_beat = mid.ticks_per_beat
    num_tracks = len(mid.tracks)

    total_notes = 0
    for track in mid.tracks:
        for msg in track:
            if msg.type == 'note_on' and msg.velocity > 0:
                total_notes += 1

    print("文件信息:")
    print(f" 文件名: {os.path.basename(file_path)}")
    print(f" 文件格式: MIDI")
    print(f" 时长: {int(duration // 60)}分{duration % 60:.2f}秒")
    print(f" 文件容量: {file_size} 字节")
    print(f" Ticks per Beat: {ticks_per_beat}")
    print(f" 轨道数: {num_tracks}")
    print(f" 音符总数: {total_notes}")

    return {
        'duration': duration,
        'file_size': file_size,
        'ticks_per_beat': ticks_per_beat,
        'num_tracks': num_tracks,
        'total_notes': total_notes
    }

result = analyze_midi("圣诞节快乐劳伦斯先生.mid")
```

文件信息

属性	值
文件名	圣诞节快乐劳伦斯先生.mid
文件格式	MIDI (Musical Instrument Digital Interface)
时长	5分28秒 (328.47秒)
文件容量	17.61 KB (18030 字节)
Ticks per Beat	384
轨道数	2
音符总数	2664

收听效果评价

MIDI文件与音频文件（如MP3、WAV）本质不同，它不包含实际的音频波形数据，而是存储一系列**音符事件信息**，包括： - 音符的音高（Pitch） - 音符的时长（Duration） - 音符的力度（Velocity） - 使用的乐器音色（Program Change）

收听效果分析：

- 音质特点：**播放时由计算机的音源（SoundFont）合成声音，音质完全取决于所使用的音色库质量。
- 优点：**
 - 文件体积极小（仅17.61 KB），便于传输和存储
 - 旋律清晰优美，能够准确表达乐曲的基本结构
 - 可以方便地修改和编辑各个声部
- 缺点：**
 - 缺乏真实钢琴的丰富音色层次和动态变化
 - 无法表现钢琴演奏中的踏板效果和细微的情感表达
 - 不同设备播放效果差异较大

总结：该MIDI文件忠实还原了《圣诞快乐劳伦斯先生》的旋律框架，但在音色表现力和情感传达方面与真实钢琴演奏存在明显差距。

题目7：图像文件分析

题目描述

分别查阅lena1bit.png、lenna_gray.png、lenna.jpg照片文件的格式、分辨率、颜色系统、容量等信息，计算它相对于原始数字图像编码容量的压缩率，并找到其中5个像素的颜色编码值。

7.1 Lenna1bit.png

代码实现

```
import os
from PIL import Image

def analyze_image(file_path):
    img = Image.open(file_path)
    file_size = os.path.getsize(file_path)

    format_name = img.format
    width, height = img.size
    mode = img.mode
    num_pixels = width * height

    if mode == '1':
        color_system = "黑白二值 (1-bit)"
        bits_per_pixel = 1
    elif mode == 'L':
        color_system = "灰度 (8-bit)"
        bits_per_pixel = 8
    elif mode == 'RGB':
        color_system = "RGB真彩色 (24-bit)"
        bits_per_pixel = 24
    elif mode == 'RGBA':
        color_system = "RGBA真彩色带透明通道 (32-bit)"
        bits_per_pixel = 32
    else:
        color_system = mode
        bits_per_pixel = 8

    print("文件信息：")
    print(f"  文件名: {os.path.basename(file_path)}")
    print(f"  文件格式: {format_name}")
    print(f"  分辨率: {width} × {height}")
    print(f"  颜色系统: {color_system}")
    print(f"  位深: {bits_per_pixel} bits/pixel")
    print(f"  像素总数: {num_pixels}")
    print(f"  文件容量: {file_size} 字节")

    original_size = num_pixels * bits_per_pixel / 8
    compression_rate = original_size / file_size

    print("\n压缩率计算过程：")
    print(f"  原始容量 = {num_pixels} × {bits_per_pixel} / 8")
    print(f"              = {original_size:.0f} 字节")
    print(f"  实际容量 = {file_size} 字节")
    print(f"  压缩率 = {original_size:.0f} / {file_size} = {compression_rate:.2f}")
    print(f"  压缩比 = {100 - (100 / compression_rate):.1f}%")

    print("\n5个像素的颜色编码值：")
    sample_positions = [(0, 0), (width//4, height//4), (width//2, height//2),
                        (3*width//4, 3*height//4), (width-1, height-1)]
    for i, (x, y) in enumerate(sample_positions):
        pixel = img.getpixel((x, y))
        if isinstance(pixel, tuple):
            hex_color = '#{0:02x}{0:02x}{0:02x}'.format(*pixel[:3])
            print(f"  像素{i+1} (位置: {x}, {y}): RGB={pixel}, 十六进制={hex_color}")
        else:
            print(f"  像素{i+1} (位置: {x}, {y}): 灰度值={pixel}")

    return {
        'format': format_name,
        'width': width,
        'height': height,
        'color_system': color_system,
        'bits_per_pixel': bits_per_pixel,
        'file_size': file_size,
        'original_size': original_size,
        'compression_rate': compression_rate
    }

result = analyze_image("Lenna1bit.png")
```

文件信息

属性	值
文件名	Lenna1bit.png
文件格式	PNG
分辨率	512 × 512
颜色系统	黑白二值 (1-bit)
位深	1 bits/pixel
像素总数	262144
文件容量	20.94 KB (21440 字节)

压缩率计算步骤

1. 原始数字图像编码容量计算公式：

原始容量 = 像素总数 × 位深 / 8

2. 代入数值计算：

原始容量 = 262144 × 1 / 8 = 32768 字节

3. 转换为更易读的单位：

32768 字节 ÷ 1024 = 32.00 KB

4. 计算压缩率：

压缩率 = 原始容量 / 实际容量 = 32768 / 21440 ≈ 1.53

5. 计算压缩比：

压缩比 = (1 - 1/1.53) × 100% ≈ 34.6%

5个像素的颜色编码值

像素编号	位置 (x, y)	灰度值	说明
像素1	(0, 0)	255	白色
像素2	(128, 128)	255	白色
像素3	(256, 256)	0	黑色
像素4	(384, 384)	255	白色
像素5	(511, 511)	255	白色

7.2 lena_gray.png

代码实现

```
result = analyze_image("lena_gray.png")
```

文件信息

属性	值
文件名	lena_gray.png
文件格式	PNG
分辨率	512 × 512
颜色系统	灰度 (8-bit)
位深	8 bits/pixel
像素总数	262144
文件容量	143.07 KB (146504 字节)

压缩率计算步骤

1. 原始数字图像编码容量计算公式：

原始容量 = 像素总数 × 位深 / 8

2. 代入数值计算：

原始容量 = 262144 × 8 / 8 = 262144 字节

3. 转换为更易读的单位：

262144 字节 ÷ 1024 = 256.00 KB

4. 计算压缩率：

压缩率 = 原始容量 / 实际容量 = 262144 / 146504 ≈ 1.79

5. 计算压缩比：

压缩比 = (1 - 1/1.79) × 100% ≈ 44.1%

5个像素的颜色编码值

像素编号	位置 (x, y)	灰度值	说明
像素1	(0, 0)	162	中等灰度
像素2	(128, 128)	174	较亮灰度
像素3	(256, 256)	106	较暗灰度
像素4	(384, 384)	141	中等灰度
像素5	(511, 511)	112	中等偏暗灰度

7.3 Lenna.jpg

代码实现

```
result = analyze_image("Lenna.jpg")
```

文件信息

属性	值
文件名	Lenna.jpg
文件格式	JPEG
分辨率	512 × 512
颜色系统	RGB真彩色 (24-bit)
位深	24 bits/pixel
像素总数	262144
文件容量	100.54 KB (102949 字节)

压缩率计算步骤

1. 原始数字图像编码容量计算公式：

原始容量 = 像素总数 × 位深 / 8

2. 代入数值计算：

原始容量 = 262144 × 24 / 8 = 786432 字节

3. 转换为更易读的单位：

786432 字节 ÷ 1024 = 768.00 KB

4. 计算压缩率：

压缩率 = 原始容量 / 实际容量 = 786432 / 102949 ≈ 7.64

5. 计算压缩比：

压缩比 = (1 - 1/7.64) × 100% ≈ 86.9%

5个像素的颜色编码值

像素编号	位置 (x, y)	RGB值	十六进制
像素1	(0, 0)	(225, 138, 128)	#e18a80

像素编号	位置 (x, y)	RGB值	十六进制
像素2	(128, 128)	(236, 153, 109)	#ec996d
像素3	(256, 256)	(180, 64, 77)	#b4404d
像素4	(384, 384)	(193, 117, 117)	#c17575
像素5	(511, 511)	(188, 72, 81)	#bc4851

题目8：视频文件分析

题目描述

查阅Pascal加法器1645.mp4文件的格式、分辨率、码率、时长、容量等信息，计算它相对于原始视频编码容量的压缩率。

代码实现

```
import os
import cv2

def analyze_video(file_path):
    cap = cv2.VideoCapture(file_path)
    file_size = os.path.getsize(file_path)

    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    fps = cap.get(cv2.CAP_PROP_FPS)
    frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    duration = frame_count / fps if fps > 0 else 0

    bitrate_kbps = (file_size * 8) / (duration * 1000) if duration > 0 else 0

    print("文件信息:")
    print(f"  文件名: {os.path.basename(file_path)}")
    print(f"  文件格式: MP4 (MPEG-4 Part 14)")
    print(f"  分辨率: {width} × {height}")
    print(f"  帧率: {fps:.2f} fps")
    print(f"  帧数: {frame_count}")
    print(f"  时长: {int(duration // 60)}分{duration % 60:.2f}秒")
    print(f"  文件容量: {file_size} 字节")
    print(f"  码率: {bitrate_kbps:.2f} kbps")

    original_size = frame_count * width * height * 24 / 8
    compression_rate = original_size / file_size

    print("\n压缩率计算过程:")
    print(f"  原始容量 = {frame_count} × {width} × {height} × 24 / 8")
    print(f"            = {original_size:.0f} 字节")
    print(f"  实际容量 = {file_size} 字节")
    print(f"  压缩率 = {original_size:.0f} / {file_size} = {compression_rate:.2f}")
    print(f"  压缩比 = {100 - (100 / compression_rate):.1f}%")

    cap.release()

    return {
        'width': width,
        'height': height,
        'fps': fps,
        'frame_count': frame_count,
        'duration': duration,
        'file_size': file_size,
        'bitrate': bitrate_kbps,
        'original_size': original_size,
        'compression_rate': compression_rate
    }

result = analyze_video("Pascal加法器1645.mp4")
```

文件信息

属性	值
文件名	Pascal加法器1645.mp4
文件格式	MP4 (MPEG-4 Part 14)
分辨率	1920 × 1080
帧率	29.97 fps
帧数	2143
时长	1分11秒 (71.50秒)
文件容量	22.16 MB (23231557 字节)
码率	2599.16 kbps

压缩率计算步骤

1. 原始视频编码容量计算公式：

原始视频编码容量 = 帧数 × 分辨率宽 × 分辨率高 × 位深 / 8

其中： - 帧数 = 2143 - 分辨率宽 = 1920 像素 - 分辨率高 = 1080 像素 - 位深 = 24 bits (RGB真彩色)

2. 代入数值计算：

原始视频编码容量 = 2143 × 1920 × 1080 × 24 / 8 = 2143 × 1920 × 1080 × 3 = 2143 × 6220800 = 13331174400 字节

3. 转换为更易读的单位：

13331174400 字节 ÷ 1024 ÷ 1024 ≈ 12713.60 MB 12713.60 MB ÷ 1024 ≈ 12.42 GB

4. 计算压缩率：

压缩率 = 原始视频编码容量 / 实际文件容量 = 13331174400 / 23231557 ≈ 573.84

5. 计算压缩比：

压缩比 = (1 - 1/573.84) × 100% ≈ 99.8%

计算结果

指标	值
原始视频编码容量	12713.60 MB (约12.42 GB)
实际文件容量	22.16 MB
压缩率	573.84
压缩比	99.8%

压缩效果分析

视频压缩率高达573.84，说明现代视频编码技术（如H.264/H.265）在保持视频质量的同时，能够实现极高的压缩效率。原始未压缩视频需要约12.42 GB的存储空间，而实际MP4文件仅需22.16 MB，压缩比达到99.8%，大大节省了存储和传输成本。

附录：各文件压缩率对比

文件	原始容量	实际容量	压缩率	压缩比
Test 1 Part 1.mp3	71.00 MB	12.88 MB	5.51	81.9%
圣诞节快乐劳伦斯先生.mid	-	17.61 KB	-	-
Lenna1bit.png	32.00 KB	20.94 KB	1.53	34.6%
lena_gray.png	256.00 KB	143.07 KB	1.79	44.1%
Lenna.jpg	768.00 KB	100.54 KB	7.64	86.9%
Pascal加法器1645.mp4	12713.60 MB	22.16 MB	573.84	99.8%

注：MIDI文件为事件编码，不存储波形数据，因此不计算压缩率。